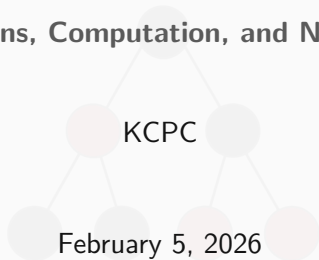
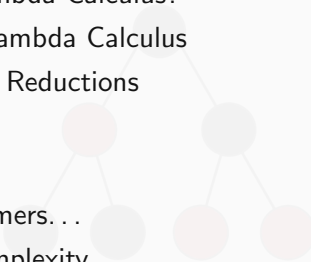


The Lambda Calculus

Functions, Computation, and Numbers



Overview

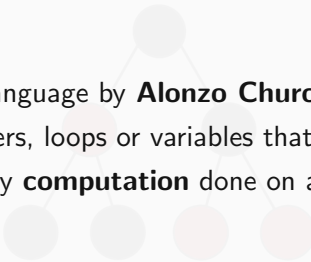
- 
- 1 What is the Lambda Calculus?
 - 2 Syntax of the Lambda Calculus
 - 3 Alpha and Beta Reductions
 - 4 Currying
 - 5 Some Purity
 - 6 We're Programmers. . .
 - 7 Some More Complexity
 - 8 Conclusion

What is the Lambda Calculus?



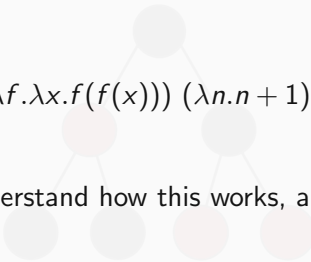
What is the Lambda Calculus?

Overview

- 
- Mathematical language by **Alonzo Church** in the 1930s
 - It has no numbers, loops or variables that store values
 - Can describe any **computation** done on a computer

What is the Lambda Calculus?

Example


$$(\lambda f. \lambda x. f(f(x))) (\lambda n. n + 1) 0$$

Evaluates to: 2

Goal of today: Understand how this works, and creating our own mathematics!

Syntax of the Lambda Calculus



What Is a Function?

- A function takes an **input**
- And produces an **output**

Mental model: A function is a machine that takes an input and produces an output.

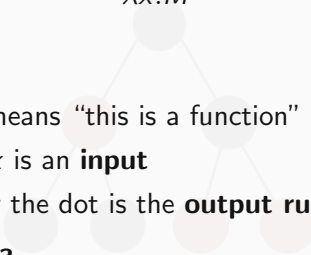
Important restriction:

- In the lambda calculus, a function takes **one input only**

Building a Function

In the lambda calculus, we build functions using **abstraction**.

$\lambda x.M$



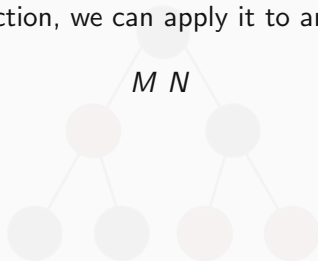
- The symbol λ means “this is a function”
- It tells us that x is an **input**
- Everything after the dot is the **output rule**

Why do we need λ ?

- To clearly say when a variable is an input
- To avoid confusion between inputs and ordinary names
- To make functions precise and unambiguous

Applying a Function

Once we have a function, we can apply it to an input.



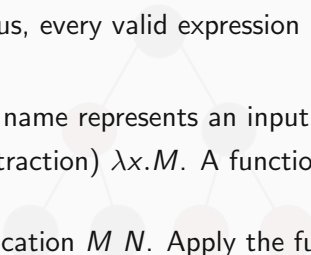
→ M is a function

→ N is its input

→ Application means “run the machine”

The Three Building Blocks

In the lambda calculus, every valid expression is called a **term**. A term is

- 
- 1 A variable x . A name represents an input.
 - 2 A function (abstraction) $\lambda x.M$. A function that takes input x and returns M .
 - 3 A function application $M N$. Apply the function M to the argument N .

Reading Terms: Parentheses and Scope

The fully parenthesised grammar is

$$M ::= x \mid (\lambda x.M) \mid (MM).$$

-
- Application groups to the left: $MNP = (MN)P$.
 - An abstraction's body extends as far right as possible:
 $\lambda x.xy = \lambda x.(xy)$.
 - $\lambda xy.M$ abbreviates $\lambda x.\lambda y.M$.

Worked example:

$$\lambda xy.x(yz) = (\lambda x.(\lambda y.(x(yz))))).$$

Its root is an abstraction; $x(yz)$ is an application.

Which Variables Are Bound?

A **binder** λx binds occurrences of x in its body. The nearest enclosing binder with the same name owns an occurrence; an occurrence with no such binder is **free**.

$$M = (\lambda x.xy)(\lambda y.yz)$$

- x in the left body is bound by λx .
- y in the left body is free; the right binder cannot reach it.
- y in the right body is bound by λy ; z is free.

Key idea: The same name can have both free and bound occurrences.

Finding Free Variables

Compute the set $FV(M)$ from the term's structure:

$$FV(x) = \{x\},$$

$$FV(MN) = FV(M) \cup FV(N),$$

$$FV(\lambda x.M) = FV(M) \setminus \{x\}.$$

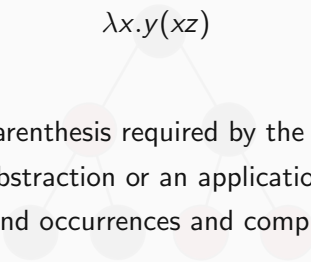
Worked example:

$$FV((\lambda x.xy)z) = (\{x, y\} \setminus \{x\}) \cup \{z\} = \{y, z\}.$$

A term is **closed** when its free-variable set is empty.

Try It Yourself: Syntax and Binding

$\lambda x.y(xz)$



- 1 Restore every parenthesis required by the grammar.
- 2 Is the root an abstraction or an application?
- 3 Identify the bound occurrences and compute the free-variable set.

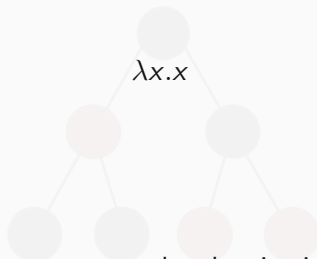
Explain your answer to a partner using scope and the three FV rules.

Example: Identity Function

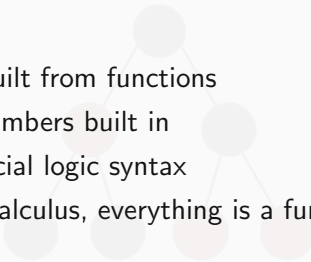
→ Input: x

→ Output: x

→ **Interpretation:** return exactly what is given



Functions, Functions Everywhere

- 
- Every term is built from functions
 - There are no numbers built in
 - There is no special logic syntax
 - In the lambda calculus, everything is a function

Alpha and Beta Reductions



Alpha Conversion

Alpha conversion is:

- The process of renaming a **bound** variable
- Used to avoid confusion or name clashes

General form:

$$\lambda x.M \longrightarrow_{\alpha} \lambda y.M[y/x]$$

Important rule:

- Renaming must not change how the input is used; choose y fresh.

Alpha Equivalence

Alpha equivalence means:

- Two functions are considered the same
- If they differ only by input names

Example: $\lambda x.x \equiv_{\alpha} \lambda y.y$

Key idea:

- Variable names are just labels
- What matters is how inputs are used

Beta Reduction

Beta reduction is where computation happens.

Applying a function means: *Replace the input variable with the argument.*

General rule:

$$(\lambda x.M) N \longrightarrow_{\beta} M[N/x]$$

Example: $(\lambda x.x + 1) 3 \longrightarrow 3 + 1 \longrightarrow 4.$

Substitute only free occurrences of x ; rename binders first if needed.

Worked Example: Avoiding Variable Capture

Reduce $(\lambda x. \lambda y. x)y$.

- 1 The argument y is free; the inner λy must not bind it.
- 2 Rename the inner binder to a fresh name z .
- 3 Substitute the argument for the free x in the body.

$$\begin{aligned} (\lambda x. \lambda y. x)y &\equiv_{\alpha} (\lambda x. \lambda z. x)y \\ &\rightarrow_{\beta} \lambda z. y. \end{aligned}$$

The result keeps y free; $\lambda y. y$ would capture it.

Try It Yourself: Safe Substitution

Reduce $(\lambda x. \lambda y. xy)y$ without capturing the free y .

- Which binder needs renaming? Choose a fresh name.
- Show the alpha-renaming and then the beta step.
- Which variable remains free in the result?

Notation: $\rightarrow_\beta$ means one beta step; $\rightarrow_\beta^*$ means zero or more steps. $\equiv_\beta$ allows steps in either direction (up to alpha-renaming).

Currying



Functions Take One Input

- In the lambda calculus, every function takes **exactly one input**
- There are no two-input or three-input functions

Problem:

- Many things in maths need more than one input
- For example: addition, multiplication

The Idea Behind Currying

Solution: A function can return another function.

- First input is given
- A new function is returned
- That new function waits for the next input

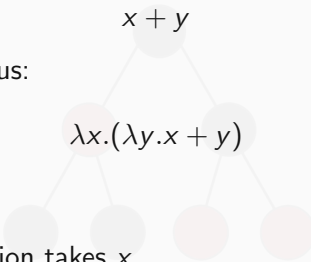
This idea is called **currying**.

Currying Explained with Addition

In ordinary maths:

$$x + y$$

In the lambda calculus:

$$\lambda x. (\lambda y. x + y)$$


- The outer function takes x
- It returns a new function
- The inner function takes y

Currying in Action

First apply the function to 1:

$$(\lambda x. \lambda y. x + y) 1 \longrightarrow \lambda y. 1 + y$$

Then apply the result to 2:

$$(\lambda y. 1 + y) 2 \longrightarrow 3$$

Conclusion:

- One input at a time
- But still behaves like a two-input function

Some Purity



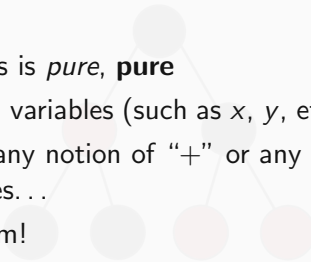
Small Issue

So far, we've used some notation without actually defining them. . .

→ $\lambda x.(\lambda y.x + y)$: what is a “+”?

→ $(\lambda y.1 + y) 2 \rightarrow 3$: what is a “2”? what is a “number”?

Purest of all Purest

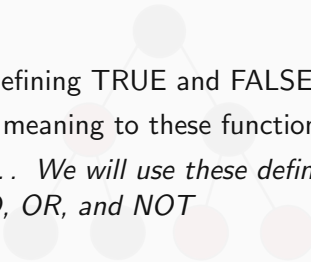
- 
- Lambda calculus is *pure*, **pure**
 - We only have λ , variables (such as x , y , etc.), and “(” and “)”
 - We don't have any notion of “+” or any of the numbers, or even truth values. . .
 - Let's create them!

We're Programmers...



We're Programmers. . .

. . . Bits!

- 
- Let's begin by defining TRUE and FALSE
 - Let's give some meaning to these functions
 - *Foreshadowing. . . We will use these definitions to create the operations AND, OR, and NOT*

We're Programmers. . .

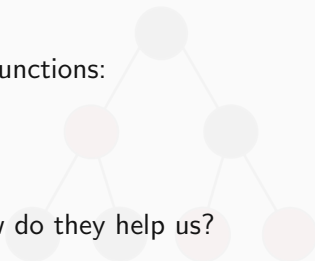
Some Hint?

Think of these two functions:

→ $\lambda x. \lambda y. x$

→ $\lambda x. \lambda y. y$

What are they? How do they help us?



Worked Example: Booleans Select a Branch

Name the two selectors

$$T = \lambda x. \lambda y. x, \quad F = \lambda x. \lambda y. y.$$

For inputs a and b :

$$\begin{aligned} T \ a \ b &\rightarrow_{\beta} (\lambda y. a) b \rightarrow_{\beta} a, \\ F \ a \ b &\rightarrow_{\beta} (\lambda y. y) b \rightarrow_{\beta} b. \end{aligned}$$

Interpretation: T chooses the first branch; F chooses the second. Both are ordinary functions, so no built-in conditional is needed.

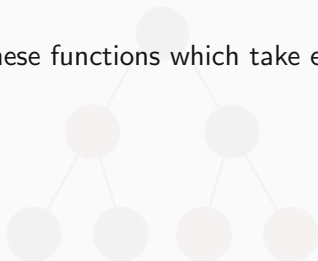
We Orn't And

Let's try to define these functions which take either a single or two parameters:

→ AND: (x, y)

→ OR: (x, y)

→ NOT: (x)



Some More Complexity

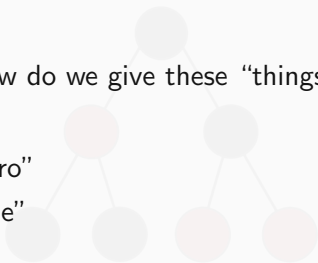


Unnaturally Natural Numbers

0, 1, 2, 3, 4, 5, ... How do we give these “things” meaning via lambda calculus?

→ Let's define “zero”

→ Let's define “one”



Some More Complexity

Oh, I Know This!

We can use *induction* to continue this pattern and to define all natural numbers. Find the successor function for your numbering system:

$$\text{SUCC}(n) = n + 1$$

Remembering Two Values: Church Pairs

Store two terms in a function that accepts a selector:

$$\langle a, b \rangle = \lambda s. sab \quad (s \text{ fresh}), \quad \text{PAIR} = \lambda a. \lambda b. \lambda s. sab.$$

Recover either value using the Boolean selectors:

$$\text{FIRST} = \lambda p. p T, \quad \text{SECOND} = \lambda p. p F.$$

$$\begin{aligned} \text{FIRST } \langle a, b \rangle &\rightarrow_{\beta}^* T \quad a \quad b \rightarrow_{\beta}^* a, \\ \text{SECOND } \langle a, b \rangle &\rightarrow_{\beta}^* F \quad a \quad b \rightarrow_{\beta}^* b. \end{aligned}$$

Use: An iterator can update a pair to remember its previous value.

Finally Some Mathematics

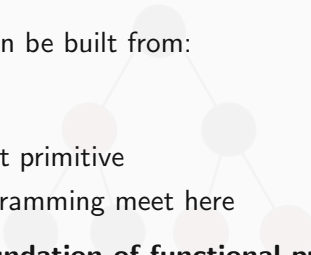
Using what we discovered and learned so far, let's finalise “our” mathematics:

- ADD: $(x, y) \mapsto x + y$
- SUB: $(x, y) \mapsto x - y$
- MUL: $(x, y) \mapsto x \times y$
- POW: $(x, y) \mapsto x^y$
- DIV: $(x, y) \mapsto x/y$ (if you can do this without looking it up, I'll give you a candy)

Conclusion



What You've Learned

- 
- Computation can be built from:
 - Functions
 - Substitution
 - Numbers are not primitive
 - Maths and programming meet here

This is also the foundation of functional programming

Try It Yourself: Syntax and Binding (Answers)

$$\lambda x. y(xz) = (\lambda x. (y(xz))).$$

- The root is an abstraction; the body $y(xz)$ is an application.
- The x in xz is bound by λx .
- The occurrences of y and z are free.

$$\begin{aligned} \text{FV}(y(xz)) &= \{y\} \cup \{x, z\} = \{x, y, z\}, \\ \text{FV}(\lambda x. y(xz)) &= \{x, y, z\} \setminus \{x\} = \{y, z\}. \end{aligned}$$

Try It Yourself: Safe Substitution (Answers)

Rename the inner binder to fresh z , then substitute:

$$\begin{aligned}(\lambda x. \lambda y. xy)y &\equiv_{\alpha} (\lambda x. \lambda z. xz)y \\ &\rightarrow_{\beta} \lambda z. yz.\end{aligned}$$

- z is bound; y remains free.
- $\lambda y. yy$ would bind the argument's y and is incorrect.

Key idea: Substitution preserves the argument's free occurrences by renaming clashing binders first.

Conclusion

Try It Yourself: Booleans (Answers)

$T = \lambda x.\lambda y.x$, $F = \lambda x.\lambda y.y$; $T\ a\ b \rightarrow_{\beta}^* a$ and $F\ a\ b \rightarrow_{\beta}^* b$.

$AND = \lambda p.\lambda q.p\ q\ F$,

$OR = \lambda p.\lambda q.p\ T\ q$,

$NOT = \lambda p.p\ F\ T$.

For example, $AND\ T\ F \rightarrow_{\beta}^* T\ F\ F \rightarrow_{\beta}^* F$.

Try It Yourself: Numbers (Answers)

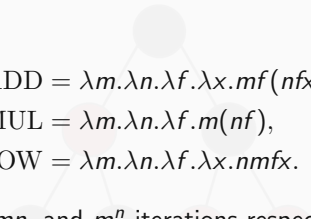
A Church numeral $\bar{n}$ applies a function n times:

$$\bar{0} = \lambda f. \lambda x. x, \quad \bar{1} = \lambda f. \lambda x. fx, \quad \bar{2} = \lambda f. \lambda x. f(fx).$$

$$\text{SUCC} = \lambda n. \lambda f. \lambda x. f(nfx)$$

Thus $\text{SUCC } \bar{n} f x \rightarrow_{\beta}^* f(\bar{n}fx)$, which applies f exactly $n + 1$ times.

Try It Yourself: Arithmetic (Answers)


$$\text{ADD} = \lambda m. \lambda n. \lambda f. \lambda x. mf(nfx),$$

$$\text{MUL} = \lambda m. \lambda n. \lambda f. m(nf),$$

$$\text{POW} = \lambda m. \lambda n. \lambda f. \lambda x. nmfx.$$

They perform $m + n$, mn , and m^n iterations respectively. SUB is truncated at zero for Church naturals; DIV additionally requires a zero-divisor convention.

Try It Yourself: Subtraction (Answers)

Using the Church pair and its projections, define

$$\text{STEP} = \lambda p. \text{PAIR} (\text{SECOND } p) (\text{SUCC}(\text{SECOND } p)).$$

It updates $\langle a, b \rangle$ to $\langle b, \text{SUCC } b \rangle$. Starting from $\langle \bar{0}, \bar{0} \rangle$:

$$\langle \bar{0}, \bar{0} \rangle \xrightarrow{\text{STEP}} \langle \bar{0}, \bar{1} \rangle \xrightarrow{\text{STEP}} \langle \bar{1}, \bar{2} \rangle \xrightarrow{\text{STEP}} \dots$$

Each state-update arrow means: apply STEP, then beta-reduce. After n updates the state is $\langle \max(n-1, 0), \bar{n} \rangle$. Hence

$$\text{PRED} = \lambda n. \text{FIRST}(n \text{ STEP } \langle \bar{0}, \bar{0} \rangle),$$

$$\text{SUB} = \lambda m. \lambda n. n \text{ PRED } m.$$

PRED maps zero to zero; SUB represents $\max(m-n, 0)$.

Try It Yourself: Division (Answers)

For $d > 0$, DIV iterates m times on a pair (q, r) initially $(0, 0)$:

- 1 Increment the remainder r .
- 2 If $r = d$, reset it to 0 and increment the quotient q .

The invariant after k steps is

$$k = qd + r, \quad 0 \leq r < d.$$

The final q is $\lfloor m/d \rfloor$. Set $\text{DIV}(m, 0) = 0$ by convention.

See the full lambda-term construction in the solutions sheet.